

CampusEats

Team

Role	Name
Member 1	Yogita gothi (20251651107)
Member 2	Janvi goud (20251651047)
Member 3	Shreya verma (20251651086)
Member 4	Sonali Choudhary (20251651088)

Services and Responsible Members

Each service is built by one member (two members each own two smaller services so the workload stays even across all four people).

Service	Data it owns	Responsible member
Account Service	Student, Address	yogita
Catalogue Service	Restaurant, MenuItem	janvi
Order Service	CartItem, Orders, OrderItem	shreya
Payment Service	Transaction, Refund	yogita
Delivery Service	Rider, DeliveryAssignment, DeliveryEvent	sonali
Notification Service	Notification	shreya

Task 1 — Capability Discovery

Capability	Responsibility	Owned Data	Service
ManageStudent Identity	Registration,loginandprofile management for students.	Logincredentials,student profiles	Accounts
ManageDelivery Addresses	Add, update and validate hostel/academic-blockdelivery addresses.	Deliveryaddresses	Accounts
BrowseRestaurants	Listandfiltercampusrestaurants available for ordering.	Restaurants	Catalogue
ExploreMenus&Availability	Viewmenuitems,pricesandreal-time item availability.	Menus,menuitems,prices, availability	Catalogue
ManageCart&Place Orders	Buildacart,placeanorderandtrackits lifecycle/status.	Carts,orders,orderitems, order status	Orders

Process Payments & Refunds	Charge student online and process refunds on cancellation.	Payment transactions, payment status, refunds	Payments
Assign & Track Delivery	Assign an available rider and track delivery progress/ETA.	Riders, delivery assignments, delivery status	Delivery
Send Order Notifications	Notify students whenever their order or delivery changes state.	Notification messages, notification status	Notifications

Capability □ Data Ownership □ Service

Manage Student Identity	□□ Login credentials, student profiles	□□ Accounts
Manage Delivery Addresses	□□ Delivery addresses	□□ Accounts
Browse Restaurants	□□ Restaurants	□□ Catalogue
Explore Menus & Availability	Menus, menu items, prices, □ availability	□ Catalogue
Manage Cart & Place Orders	Carts, orders, order items, order □ status	□ Orders
Process Payments & Refunds	Payment transactions, payment □ status, refunds	□ Payments
Assign & Track Delivery	Riders, delivery assignments, □ delivery status	□ Delivery
Send Order Notifications	Notification messages, notification □ status	□ Notifications

CampusEats

Service Contracts

ACCOUNTS CONTRACT

Operation	Output	Errors
authenticate(email, password)	{studentId, sessionToken}	INVALID_CREDENTIALS, ACCOUNT_LOCKED
getProfile(studentId)	{name,email, hostelBlock}	STUDENT_NOT_FOUND
validateAddress(studentId, addressId)	{valid,addressLine, block}	ADDRESS_NOT_FOUND, FORBIDDEN
updateAddress(studentId,addressId, addressDetails)	{addressId, updated}	INVALID_ADDRESS, FORBIDDEN

CATALOGUE CONTRACT

Operation	Output	Errors
listRestaurants(areaFilter)	RestaurantSummary[]	INVALID_FILTER
getMenu(restaurantId)	MenuItem[]	RESTAURANT_NOT_FOUND
checkItem(itemId, quantity)	{available, unitPrice}	ITEM_NOT_FOUND, INSUFFICIENT_STOCK
searchMenu(keyword, restaurantId)	MenuItem[]	INVALID_KEYWORD

ORDERS CONTRACT

Operation	Output	Errors
addToCart(studentId, itemId, quantity)	CartSummary	ITEM_UNAVAILABLE, INVALID_QUANTITY

placeOrder(studentId,items,addressId, paymentMethod)	{orderId, status, total}	EMPTY_CART, INVALID_ADDRESS, ITEM_UNAVAILABLE, PAYMENT_DECLINED
getOrder(studentId, orderId)	OrderDetails	ORDER_NOT_FOUND, FORBIDDEN
cancelOrder(studentId, orderId)	CancelResult	ORDER_NOT_FOUND, CANNOT_CANCEL
PAYMENTS CONTRACT		
Operation	Output	Errors
charge(orderId,studentId,amount, paymentMethod)	{transactionId, status}	PAYMENT_DECLINED, INVALID_AMOUNT
refund(orderId, transactionId, reason)	{refundId, status}	TRANSACTION_NOT_FOUN D,REFUND_NOT_ELIGIBLE
getPaymentStatus(orderId)	{status, amount}	TRANSACTION_NOT_FOUN D

DELIVERY CONTRACT		
Operation	Output	Errors
assignRider(orderId,restaurantId, addressId)	{riderId, eta}	NO_RIDER_AVAILABLE, INVALID_ADDRESS
getDeliveryStatus(orderId)	{status,riderLocation, eta}	ASSIGNMENT_NOT_FOUND
updateDeliveryStatus(orderId, event)	{status, updated}	ASSIGNMENT_NOT_FOUND, INVALID_EVENT

NOTIFICATIONS CONTRACT		
Operation	Output	Errors
sendOrderUpdate(orderId,studentId, event)	{notificationId, status}	INVALID_EVENT, STUDENT_NOT_FOUND
sendDeliveryUpdate(orderId,studentId, event)	{notificationId, status}	INVALID_EVENT, STUDENT_NOT_FOUND

getNotificationStatus(notificationId)	{status, sentAt}	NOTIFICATION_NOT_FOUND
---------------------------------------	------------------	------------------------

Task 4 — Full Specification of placeOrder

placeOrder is the central operation of the Order Service. It turns a student's cart into a confirmed order by checking the address, checking item availability, charging the payment, and (once payment succeeds) asking the Delivery Service to assign a rider.

Operation signature

placeOrder(studentId, items[], addressId, paymentMethod) → { orderId, status, total }

Inputs

Field	Type / Notes
studentId	String, required. The student placing the order (must match the logged-in session).
items[]	Array, required, must not be empty. Each entry: { itemId, quantity }.
addressId	String, required. Delivery address id belonging to the student.
paymentMethod	Enum, required. One of CARD, UPI, WALLET, COD.

Outputs

Field	Type / Notes
orderId	String. Unique id of the newly created order.
status	String. Order status after processing, e.g. PLACED.
total	Decimal. Final order amount (sum of item prices x quantity, using live catalogue prices).

Error cases

Field	Type / Notes
EMPTY_CART	items[] is empty or missing.
INVALID_ADDRESS	addressId does not exist, or does not belong to studentId (checked via Account Service validateAddress).
ITEM_UNAVAILABLE	One or more items are out of stock or no longer available (checked via Catalogue Service checkItem).
PAYMENT_DECLINED	The Payment Service charge() call failed or was declined.

Internal steps (hidden from callers)

The caller only ever sees the inputs, the output, and the four error codes above. Everything below happens inside the Order Service and is never exposed through the contract:

- 1. Reject immediately with EMPTY_CART if items[] is empty.
- 2. Call Account Service validateAddress(studentId, addressId). If it fails, return INVALID_ADDRESS.
- 3. For every item in items[], call Catalogue Service checkItem(itemId, quantity) to confirm stock and fetch the current price — the price is never taken from the client request, to stop price tampering. If any item fails, return ITEM_UNAVAILABLE.
- 4. Compute the order total on the server from the prices just fetched.
- 5. Create an Order row and matching OrderItem rows with status = PENDING, inside a single local database transaction.
- 6. Call Payment Service charge(orderId, studentId, total, paymentMethod). If it fails, set the order status to CANCELLED and return PAYMENT_DECLINED.
- 7. On a successful charge, update the order status to PLACED and call Delivery Service assignRider(orderId, restaurantId, addressId) in the background — a rider assignment delay does not block the response to the student.
- 8. Call Notification Service sendOrderUpdate(orderId, studentId, "PLACED") in the background (fire-and-forget).
- 9. Return { orderId, status, total } to the caller.
- 10. Duplicate-submission handling (idempotency), retry/timeout limits when calling other services, and the exact table structure are internal details — a caller only needs the contract above to use placeOrder correctly.

Task 6 — Validation Against the Five Properties of a Service

Each service is checked against: (1) Reachable — can other services call it through a fixed address/contract; (2) Self-contained — does it hold all the data and logic it needs to do its job; (3) Has a contract — does it publish a clear operation list with inputs/outputs/errors; (4) Independent — can it be built, deployed, and changed on its own; (5) Loosely coupled — does it depend on the internal design of other services, or only on their contracts.

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Notes / Fix if failing
Account Service	Yes	Yes	Yes	Yes	Yes	Owns Student and Address tables only. Exposes authenticate, getProfile, validateAddress, updateAddress through its contract. No other service touches its tables directly.
Catalogue Service	Yes	Yes	Yes	Yes	Yes	Owns Restaurant and MenuItem tables. All price/availability checks go through checkItem, not direct table access.
Order Service	Yes	Partially	Yes	Partially	Partially	Order Service depends on Account (address), Catalogue (price/stock) and Payment (charge) to complete a single placeOrder call, so it is not fully self-contained or independent. This is acceptable because it is an orchestrator, but if Payment or Catalogue is down, placeOrder cannot finish. Fix: keep the calls behind timeouts/retries and fail safely (mark order CANCELLED) rather than blocking forever.
Payment Service	Yes	Yes	Yes	Yes	Yes	Owns Transaction and Refund tables. Charges and refunds are done only through its own contract; no other service reaches into its tables.
Delivery Service	Yes	Yes	Yes	Yes	Mostly	Owns Rider, DeliveryAssignment, DeliveryEvent tables. It only receives an orderId reference, so it stays loosely coupled;

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Notes / Fix if failing
						it does not know about Payment or Catalogue at all.
Notification Service	Yes	Yes	Yes	Yes	Yes	Owens only the Notification table. It is called at the end of a flow (fire-and-forget) by Order/Delivery, so failures here do not block other services.

Overall note: five of the six services fully satisfy all five properties because each owns a small, clear slice of data and is only ever reached through its published operations. The Order Service is the one exception — as the orchestrator of `placeOrder`, it must call Account, Catalogue and Payment to finish a single request, so it is only “partially” self-contained and independent. This is a normal and expected trade-off for an orchestrating service, and it is kept loosely coupled by only ever calling the other services' published contracts (never their tables directly), and by handling their failures with its own error codes (`INVALID_ADDRESS`, `ITEM_UNAVAILABLE`, `PAYMENT_DECLINED`) instead of crashing.